

Natural Language Processing

Text Classification & POS Tagging with BERTS

Jan 2026

**Maria Schoinaki
Electra Papadopoulou
Leonidas Kontogiannis**

You can view the full code implementation for the **text classification** in the following link:



You can view the full code implementation for the **POS tagger** in the following link:



All members have contributed equally in all tasks, markdown and report writing, synchronously throughout the assignment.

Tweet Sentiment Classification with BERTs

For this assignment we used the Twitter Sentiment Analysis dataset from Kaggle. This dataset consists of ~100.000 tweets (sentiment texts), the id of each tweet, and the sentiment (0 or 1), and it was used as the basis for training (70%), development (15%), and testing (15%) of the sentiment classifiers. The same dataset had been used for previous assignments, thus we will be brief in summarizing the preprocess and stage. For more details, you can read the report [here](#).

Data Preprocessing/ Cleaning

To preprocess the dataset, we use the pretrained **DistilRoBERTa tokenizer** provided by the Hugging Face Transformers library. The model was pretrained on raw text.

- **Text Preparation:**

Each dataset split is converted into a Hugging Face Dataset object containing raw text and labels. The tokenizer performs subword tokenization, adds special tokens, and converts text into numerical token IDs. Input sequences are truncated to fit the model's maximum length.

- **Batch Padding and Formatting:**

Dynamic padding is applied at training time using a data collator, which pads each batch to the length of the longest sequence in that batch. The tokenized datasets are then formatted as PyTorch tensors containing `input_ids`, `attention_mask`, and `label` fields.

Experimental Setup

For Exercise 1 we reuse exactly the same dataset and splits from exercise 11 of Part 2, exercise 4 of Part 3, exercise 1 of Part 4, exercise 2 of Part 5, but we handle preprocessing differently.

- **Corpus:** twitter-sentiment-analysis2 (Kaggle), with ~100K tweets labeled with binary sentiment (0 = Negative, 1 = Positive).
- **Splits:** 70% train, 15% development, 15% test, stratified by label, using `random_state=2025`.
- **Preprocessing:** Text Preparation and Batch Padding and Formatting, as described above.

Baseline Classifiers

We consider all our previous baselines, as requested:

- **TF-IDF + Logistic Regression (best model from Exercise 11 part 2):**
TF-IDF n-gram features with SelectKBest (mutual information, $k = 30,000$), followed by Logistic Regression ($C = 1$).
- **MLP (TF-IDF + SVD) (From Exercise 4 in part 3):**
TF-IDF features reduced to 500 dimensions using SVD, followed by a single hidden layer MLP (512 units, ReLU, dropout = 0.7) and a linear output layer.
- **BiLSTM + Word Embeddings (From Exercise 1 in part 4):**
Trainable word embeddings passed through a 2-layer BiLSTM (hidden size = 128 per direction), with global average pooling and a linear classifier.
- **CNN (Stacked Convolutional Encoder) (From Exercise 2 in part 5):**
Pretrained word embeddings (fine-tuned, dropout = 0.4) processed by two stacked convolutional blocks with parallel 1D kernels (sizes 2 and 3, 64 channels), residual connections, and layer normalization, followed by global max pooling and an MLP classifier.

Model Architecture

Network architecture

The final best configuration is a parameter-efficient BERT-style text classifier built on top of DistilRoBERTa with frozen encoder layers and LoRA adapters.

Transformer encoder (DistilRoBERTa):

- Base model: `distilroberta-base`
- 6 transformer encoder layers
- All encoder layers frozen during training (`freeze = 6`)
- Tokenization performed using the pretrained DistilRoBERTa tokenizer with truncation

Freezing the encoder allows the model to retain pretrained linguistic knowledge while reducing the number of trainable parameters and stabilizing training on limited labeled data.

LoRA adaptation:

- LoRA adapters are put into the **query** and **value** projection matrices of the self-attention layers
- LoRA rank: $r = 8$
- LoRA scaling factor: $\alpha = 32$
- LoRA dropout: 0.2
- Only LoRA parameters and the classification head are trainable

Classifier head:

- The pooled sentence representation produced by DistilRoBERTa is passed directly to a linear classification head.
- Output dimension: 2 (Negative, Positive)
- Trained using cross-entropy loss.

Training

The model is trained using mini-batches of tokenized sentences. Each training epoch consists of a full pass over the training set, computing the forward pass, loss, and gradients, followed by an optimizer update. Training is implemented using the Hugging Face `Trainer` API. Evaluation is performed at the end of every epoch on the development set, computing **accuracy** and **macro-F1**, with macro-F1 used as the model selection criterion. Early stopping is applied with a patience of 2 epochs: if development macro-F1 does not improve, training is halted automatically. The parameters corresponding to the best development macro-F1 are restored at the end of training to ensure robust generalization.

Hyperparameter Tuning (With Frozen Layers)

We conduct a hyperparameter search over several **LoRA fine-tuning configurations**, varying:

- Learning rate
- LoRA dropout

- Number of frozen transformer layers

Rather than performing full fine-tuning of the transformer, we focus on **parameter-efficient adaptation** by keeping most of the model frozen and tuning only LoRA parameters and the classifier head. By doing this, we can reduce the computational cost as well.

Training Procedure

For each configuration:

1. A new DistilRoBERTa sequence classification model is initialized.
2. Encoder layers are frozen according to the configuration.
3. LoRA adapters are put into the attention layers.
4. The model is trained for up to 20 epochs with early stopping (patience = 2).
5. Development macro-F1 is used as the primary selection metric.
6. The best-performing model checkpoint is retained.

All configurations are trained independently, and their development macro-F1 scores are recorded for comparison.

Best Model and Hyperparameters

The best-performing configuration was obtained with the following hyperparameters:

- Base model: `distilroberta-base`
- Frozen encoder layers: 6 (fully frozen)
- LoRA rank: 8
- LoRA dropout: 0.2
- Learning rate: 1e-3
- Batch size: 64
- Optimizer: default optimizer provided by the Hugging Face Trainer
- Maximum epochs: 20 (early stopping applied)

The **best model's macro F1 score** on the development set was **~0.84**.

Notes on implementation

- Only LoRA parameters and the classification head are trainable, resulting in a dramatic reduction in trainable parameters compared to full fine-tuning.
- Early stopping and automatic restoration of the best checkpoint ensure stable evaluation.
- Training logs include epoch-level metrics for loss, accuracy, and macro-F1, enabling inspection of convergence and generalization behavior.

Loss Curves

After training, we analyze the behavior of the best model across epochs by visualizing the training history:



- **Train vs. Dev Loss:** The curves help determine whether the model continues learning (**training loss decreasing**), whether it generalizes well (**dev loss follows train loss without diverging**) and whether overfitting occurs (**dev loss increases while train loss continues to drop**).

Evaluation

We report per-class **Precision (P)**, **Recall (R)**, **F1**, and **PR-AUC** and we also include **Macro-Averaged P/R/F1/PR-AUC**. Models are **trained** on *train* dataset, **tuned** in *dev* dataset and **evaluated** in *test* dataset.

Systems Compared

- **Majority baseline** (*dummy classifier-most_frequent*): Always predicts the training set's majority class.
- **LR (TF-IDF) tuned**: Logistic Regression with TF-IDF word n-grams (*uni, bi, tri*).
- **MLP (TF-IDF) tuned**: Multi-Layered Perceptron with SVD features.
- **BiLSTM** (word embedding) **tuned**: Bidirectional LSTM using pretrained word embeddings.
- **CNN** (word embedding) **tuned**: Stacked CNN using pretrained word embeddings.
- **BERT + LoRA** (parameter-efficient) **tuned**: Pretrained BERT-based encoder with frozen backbone and LoRA adapters trained for sequence classification.

Results

We provide three tables (*train, dev, test*). Each table has one row per model and four columns per class (*P, R, F1, PR-AUC*), followed by *Macro-P, Macro-R, Macro-F1, Macro-PR-AUC*.

Training Set

=== TRAIN RESULTS ===

	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435364	0.564636	1.000000	0.721747	0.564636	0.282318	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.800657	0.831846	0.815953	0.884331	0.866329	0.840309	0.853121	0.934591	0.833493	0.836077	0.834537	0.909461
2	MLP	0.825968	0.690601	0.752243	0.860903	0.788201	0.887804	0.835043	0.906844	0.807084	0.789202	0.793643	0.883873
3	BiLSTM	0.846344	0.881285	0.863461	0.929839	0.905463	0.876642	0.890819	0.959860	0.875903	0.878963	0.877140	0.944850
4	CNN	0.802078	0.859901	0.829983	0.898172	0.885628	0.836403	0.860312	0.942985	0.843853	0.848152	0.845148	0.920578
5	BERT + LoRA	0.859053	0.849468	0.854234	0.930197	0.884922	0.892535	0.888713	0.954604	0.871988	0.871002	0.871473	0.942400

- The **majority baseline** reaches Macro-F1 ≈ 0.36 and Macro PR-AUC ≈ 0.50 (*constant prediction yields poor F1 but PR-AUC near the class prior*).
- TF-IDF + Logistic Regression, MLP, BiLSTM, and CNN all achieve strong training performance, with the BiLSTM and CNN benefiting from word-order awareness and contextual modeling. However, the BERT + LoRA model achieves the strongest overall training performance, reaching a **Macro-F1 of 0.871** and **Macro PR-AUC of 0.942**.

Development Set

=== DEV RESULTS ===

	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435391	0.564609	1.000000	0.721725	0.564609	0.282304	0.500000	0.360863	0.500000
1	TF-IDF+LR	0.735910	0.763859	0.749624	0.814233	0.812409	0.788616	0.800336	0.881390	0.774160	0.776238	0.774980	0.847811
2	MLP	0.707028	0.594640	0.645982	0.739800	0.721544	0.809991	0.763214	0.822055	0.714286	0.702315	0.704598	0.780927
3	BiLSTM	0.737275	0.757459	0.747231	0.819064	0.808825	0.791741	0.800192	0.880489	0.773050	0.774600	0.773711	0.849777
4	CNN	0.719030	0.765918	0.741734	0.809184	0.809821	0.769076	0.788923	0.876072	0.764426	0.767497	0.765328	0.842628
5	BERT + LoRA	0.818111	0.809342	0.813703	0.895644	0.854181	0.861242	0.857697	0.931772	0.836146	0.835292	0.835700	0.913708

- **Baseline:** Macro F1 ≈ 0.3609 , Macro PR-AUC = 0.5000
- **TF-IDF + LR:** Macro F1 ≈ 0.775 , Macro PR-AUC ≈ 0.848
- **MLP (TF-IDF):** Macro F1 ≈ 0.705 , Macro PR-AUC ≈ 0.781
- **BiLSTM (word embeddings):** Macro F1 ≈ 0.774 , Macro PR-AUC ≈ 0.850
- **CNN (word embeddings):** Macro F1 ≈ 0.765 , Macro PR-AUC ≈ 0.843
- **BERT + LoRA:** Macro F1 ≈ 0.836 , Macro PR-AUC ≈ 0.914

Observation:

BERT + LoRA clearly outperforms all other models on the development set by a large margin. Notably, this performance is achieved without full fine-tuning of the transformer encoder, demonstrating the effectiveness of LoRA-based adaptation under frozen BERT representations.

Test Set

=== TEST RESULTS ===

	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435362	0.564638	1.000000	0.721749	0.564638	0.282319	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.731323	0.754058	0.742517	0.817475	0.805710	0.786397	0.795937	0.880550	0.768517	0.770228	0.769227	0.849012
2	MLP	0.707851	0.589587	0.643329	0.746715	0.719665	0.812375	0.763215	0.821697	0.713758	0.700981	0.703272	0.784206
3	BiLSTM	0.740409	0.751693	0.746009	0.819974	0.806525	0.797060	0.801765	0.879283	0.773467	0.774377	0.773887	0.849628
4	CNN	0.725971	0.756927	0.741126	0.814234	0.806471	0.779991	0.793010	0.876709	0.766221	0.768459	0.767068	0.845472
5	BERT + LoRA	0.824520	0.802297	0.813257	0.898605	0.850665	0.868343	0.859413	0.927978	0.837593	0.835320	0.836335	0.913291

- **Baseline:** Extremely poor for class 0 ($F1 = 0$), reasonable for class 1; overall Macro F1 ≈ 0.36 .
- **TF-IDF + LR:** Solid and balanced performance, Macro F1 ≈ 0.769 , Macro PR-AUC ≈ 0.849 .
- **MLP:** Worse than TF-IDF + LR and BiLSTM, Macro F1 ≈ 0.703 .
- **BiLSTM:** Comparable to TF-IDF + LR, balanced precision and recall across both classes; Macro F1 ≈ 0.774 , Macro PR-AUC ≈ 0.850 .
- **CNN:** Close to BiLSTM, Macro F1 ≈ 0.767 , Macro PR-AUC ≈ 0.845 .
- **BERT + LoRA: Best overall performance**, with **Macro F1 ≈ 0.836** and **Macro PR-AUC ≈ 0.913** .

LLM Prompting

In addition to the BERT classifier, we conducted an exploratory experiment using a **large language model** (LLM) for sentiment classification via prompt-based inference. The approach constructs a few-shot prompt by randomly sampling labeled tweets from the training set and formatting them as input–output demonstrations. Each prompt instructs the LLM to classify tweet sentiment (0 for Negative, 1 for Positive). For evaluation, a small subset of 10 tweets is randomly sampled from the test set and individually queried using the same instruction format, without revealing their gold labels in the prompt.

Our results from this are as follows:

	Tweet	Gold	ChatGPT
0	@BlaqueBeautiful yeah it is a strong word dont...	0	0
1	@Catholiccitizen rest well	1	1
2	@andrewmilton wow i see you left a barbershop ...	1	1
3	@bygbaby lol Sleep well hun. I am following s...	1	1
4	@annkogut annaaaa! fez twitter quando, menin...	1	1
5	@bouska I want a #Noh8 Photo! #jealous!	0	1
6	@alagerst yeah, you look kinda plaguey in gree...	0	0
7	:)...bored...just came back from a bike ride b...	0	0
8	#howarewe 3/10	0	0
9	@AnInspiredMind Your welcome	1	1

The predicted labels are compared against the ground-truth annotations. On a subset of 10 test examples, the LLM achieves an accuracy of **0.90** and a **Macro-F1 score of 0.90**. While this experiment is limited in

scale and doesn't include fine-tuning, we can tell that large LLMs can perform competitive sentiment classification using only natural-language instructions and a small number of labeled examples.

POS Tagging

For this assignment we used the UD Ancient Greek PROIEL treebank from Universal Dependencies. The corpus contains morphologically and syntactically annotated **Ancient Greek** texts. After loading the dataset, we obtained **15,016** sentences for training, **1,019** sentences for development, and **1,047** sentences for testing. Each sentence consists of a sequence of tokens paired with their corresponding Universal POS tags (UPOS). The same dataset had been used for previous assignments, thus we will be brief in summarizing the preprocess and stage. For more details, you can read the report [here](#).

Data Preprocessing

For the CNN model, sentences are processed as padded sequences: local context is captured implicitly by convolutional kernels of varying widths. At sentence boundaries, missing context is filled with the special <PAD> token.

The following table summarizes **key statistics** for the training, development, and test subsets of the **Ancient Greek PROIEL** treebank:

```
TRAIN stats:
sentences:      15016
tokens:         187039
avg sent length: 12.46
median sent len: 10.00
vocab size:     30349
number of tags: 13
most common tags:
  VERB: 34517
  NOUN: 31424
  DET: 28070
  ADV: 20288
  PRON: 19756
  ADP: 13812
  CCONJ: 12117
  ADJ: 10361
  PROPN: 7745
  SCONJ: 3610
```

```
DEV stats:
sentences:      1019
tokens:         13652
avg sent length: 13.40
median sent len: 11.00
vocab size:     4541
number of tags: 13
most common tags:
  VERB: 2671
  NOUN: 2185
  DET: 2082
  PRON: 1446
  ADV: 1378
  CCONJ: 978
  ADP: 922
  ADJ: 729
  PROPN: 618
  SCONJ: 257
```

```
TEST stats:
sentences:      1047
tokens:         13314
avg sent length: 12.72
median sent len: 11.00
vocab size:     4495
number of tags: 13
most common tags:
  VERB: 2494
  NOUN: 2209
  DET: 2006
  PRON: 1427
  ADV: 1416
  CCONJ: 1011
  ADP: 885
  ADJ: 752
  PROPN: 494
  AUX: 248
```

We trained **Word2Vec** embeddings directly on the **training corpus**. Only the word sequences were used as input for Word2Vec, and we employed a **skip-gram architecture** with a vector dimension of **100**, a

window size of 5, and a minimum frequency threshold of one. The Word2Vec model was trained for ten epochs. This produced distributional word vectors for all words seen in the training data.

Our Model

The POS tagger is based on **pretrained BERT (Bidirectional Encoder Representations from Transformers)** fine-tuned for token classification. Each sentence is tokenized into subword units using BERT's tokenizer, and words are aligned with labels so that predictions are made at the token level. Tokens corresponding to subword splits or padding positions are ignored during loss computation by assigning an **ignore_index** label. To control model capacity and improve stability, we experiment with freezing the lower encoder layers of BERT while fine-tuning the remaining layers. In addition, we add a small task-specific multilayer perceptron (MLP) head on top of BERT's token representations. This allows the model to adapt pretrained contextual embeddings to the POS tagging task while limiting the number of trainable parameters.

Training uses the sequence of BERT embeddings, which are passed through the standard transformer layers, producing contextualized representations for each token. The contextualized token representations are passed through a small task-specific MLP head, followed by a final linear classifier that maps embeddings to POS tags. **Cross-entropy loss** is computed only for non-ignored positions, and optimization is performed using AdamW (Trainer from Hugging Face defaults to it). Model performance is monitored after each epoch using development-set macro-F1, micro-F1, and accuracy, with early stopping applied to prevent overfitting. The checkpoint corresponding to the best development macro-F1 is restored at the end of training to ensure stable generalization.

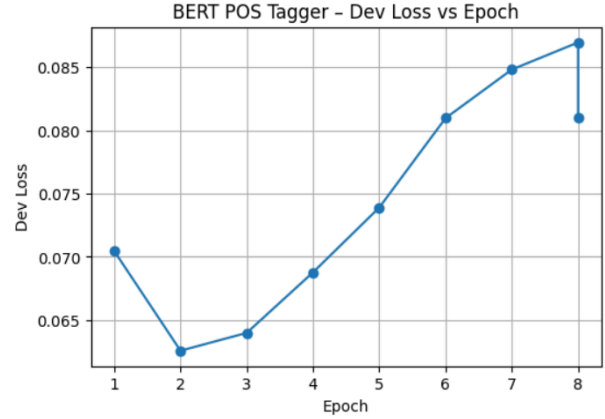
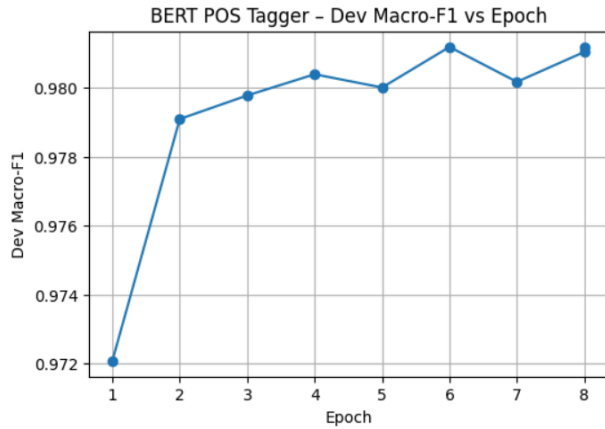
Fine Tuning and Training

To identify the best BERT-based POS tagger, we conducted a small hyperparameter search over learning rate, batch size, weight decay, the number of frozen BERT encoder layers, and the size and depth of a task-specific MLP head. Each combination was trained independently on the training set and evaluated on the development set using macro-F1 as the main selection criterion. Training was performed for up to **10 epochs**, with early stopping patience of 2 epochs. Metrics were computed ignoring padding tokens, and each run recorded both training and development performance curves to assess convergence.

The best-performing configuration was:

- **Model:** Pretrained BERT
- **Learning rate:** 2e-5
- **Batch size:** 8
- **Weight decay:** 0.01
- **Number of epochs:** 10 (early stopping with patience 2)
- **Optimizer:** AdamW
- **Evaluation metric:** development macro-F1

This configuration achieved a **development macro-F1 of 0.9812** and **accuracy of 0.9832**. The training procedure employed **full-sequence tokenization with truncation at 512 tokens** and a **DataCollatorForTokenClassification** to batch inputs of varying lengths efficiently.



The development macro-F1 initially improves but later stagnates or decreases, while development loss begins to rise, indicating overfitting if we were to train for too many epochs. However, by selecting the epoch with the best development macro-F1, the final model corresponds to an early epoch before severe overfitting sets in.

Evaluation

Once again, we create three tables that summarize the training, development, and test set performance of all POS-tagging models across all UPOS categories. The results that interest us are as follows:

=== TRAIN RESULTS (updated) ===

	Model	P(AD3)	R(AD3)	F1(AD3)	PR-AUC(AD3)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCON3)	PR-AUC(SCON3)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9845	0.9908	0.9876	0.9759	0.9765	0.9940	0.9852	0.9711	0.9726	...	0.8797	0.7775	0.9996	0.9992	0.9994	0.9989	0.9671	0.9665	0.9664	0.9384
1	MLP (best config)	0.9948	0.9880	0.9914	0.9994	0.9946	0.9981	0.9964	0.9999	0.9838	...	0.9730	0.9951	0.9996	0.9999	0.9997	1.0000	0.9897	0.9905	0.9901	0.9889
2	BILSTM	0.9927	0.9887	0.9907	0.9995	0.9950	0.9986	0.9967	0.9999	0.9941	...	0.9730	0.9958	0.9995	0.9984	0.9990	1.0000	0.9892	0.9913	0.9902	0.9991
3	BILSTM-char	0.9896	0.9940	0.9918	0.9994	0.9953	0.9980	0.9966	0.9998	0.9853	...	0.9694	0.9917	0.9996	0.9992	0.9994	1.0000	0.9892	0.9894	0.9893	0.9985
4	BILSTM-TA	0.9976	0.9986	0.9981	1.0000	0.9968	0.9999	0.9983	1.0000	0.9979	...	0.9916	0.9995	0.9999	0.9999	0.9999	1.0000	0.9964	0.9962	0.9963	0.9999
5	CNN	0.9948	0.9932	0.9940	0.9997	0.9975	0.9988	0.9981	1.0000	0.9919	...	0.9848	0.9983	0.9992	0.9972	0.9982	1.0000	0.9929	0.9918	0.9923	0.9995
6	CNN-char	0.9790	0.9875	0.9833	0.9974	0.9939	0.9943	0.9941	0.9997	0.9682	...	0.9410	0.9848	0.9985	0.9989	0.9987	1.0000	0.9783	0.9780	0.9781	0.9959
7	CNN-TA	0.9939	0.9989	0.9964	1.0000	0.9969	0.9998	0.9983	1.0000	0.9976	...	0.9888	0.9989	1.0000	0.9999	1.0000	1.0000	0.9944	0.9954	0.9948	0.9998
8	BERT (Ancient-Greek-BERT)	0.9993	0.9994	0.9994	1.0000	1.0000	0.9999	1.0000	1.0000	0.9996	...	0.9994	1.0000	0.9999	1.0000	0.9999	1.0000	0.9995	0.9995	0.9995	1.0000

=== DEV RESULTS (updated) ===

	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9774	0.7133	0.8247	0.7125	0.9683	0.9935	0.9807	0.9624	0.9556	...	0.8419	0.7201	0.7957	0.9989	0.8858	0.7950	0.9489	0.9039	0.9211	0.8627
1	MLP (best config)	0.9171	0.7284	0.8119	0.8975	0.9860	0.9946	0.9903	0.9993	0.9591	...	0.9165	0.9726	0.8359	0.9955	0.9087	0.9852	0.9505	0.9251	0.9356	0.9750
2	BiLSTM	0.9262	0.7407	0.8232	0.9005	0.9871	0.9946	0.9908	0.9993	0.9749	...	0.9246	0.9725	0.8429	0.9925	0.9116	0.9815	0.9563	0.9256	0.9379	0.9749
3	BiLSTM-char	0.8053	0.8738	0.8382	0.9398	0.9882	0.9957	0.9919	0.9996	0.9623	...	0.9130	0.9701	0.9603	0.9880	0.9740	0.9969	0.9532	0.9534	0.9530	0.9869
4	BiLSTM-TA	0.9314	0.7449	0.8277	0.8969	0.9850	0.9957	0.9903	0.9992	0.9648	...	0.9167	0.9727	0.8373	0.9959	0.9097	0.9836	0.9536	0.9249	0.9371	0.9754
5	CNN	0.9027	0.7380	0.8121	0.8886	0.9903	0.9946	0.9924	0.9992	0.9578	...	0.9198	0.9737	0.8485	0.9854	0.9118	0.9822	0.9481	0.9266	0.9359	0.9734
6	CNN-char	0.8351	0.8820	0.8579	0.9478	0.9913	0.9902	0.9908	0.9996	0.9580	...	0.9201	0.9753	0.9678	0.9891	0.9783	0.9975	0.9551	0.9577	0.9562	0.9879
7	CNN-TA	0.8321	0.8871	0.8634	0.9477	0.9871	0.9967	0.9919	0.9996	0.9772	...	0.9243	0.9796	0.9421	0.9933	0.9670	0.9969	0.9561	0.9556	0.9555	0.9881
8	BERT (Ancient-Greek-BERT)	0.9596	0.9451	0.9523	0.9806	0.9903	0.9957	0.9930	0.9994	0.9808	...	0.9669	0.9891	0.9985	0.9959	0.9972	0.9998	0.9805	0.9819	0.9812	0.9945

=== TEST RESULTS (updated) ===

	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9730	0.7660	0.8571	0.7585	0.9723	0.9932	0.9827	0.9662	0.9746	...	0.8904	0.7953	0.7987	0.9976	0.8871	0.7972	0.9523	0.9197	0.9325	0.8811
1	MLP (best config)	0.9211	0.7606	0.8332	0.9094	0.9910	0.9955	0.9932	0.9998	0.9649	...	0.9459	0.9839	0.8394	0.9932	0.9098	0.9817	0.9496	0.9323	0.9393	0.9765
2	BiLSTM	0.9154	0.7766	0.8403	0.9096	0.9877	0.9977	0.9927	0.9995	0.9798	...	0.9367	0.9833	0.8431	0.9868	0.9093	0.9745	0.9555	0.9335	0.9425	0.9748
3	BiLSTM-char	0.7845	0.9003	0.8384	0.9449	0.9966	0.9977	0.9972	0.9998	0.9694	...	0.9423	0.9839	0.9584	0.9892	0.9736	0.9964	0.9547	0.9548	0.9542	0.9878
4	BiLSTM-TA	0.9320	0.7832	0.8512	0.9102	0.9855	0.9977	0.9916	0.9998	0.9806	...	0.9517	0.9861	0.8388	0.9928	0.9093	0.9781	0.9541	0.9345	0.9428	0.9750
5	CNN	0.8986	0.7899	0.8408	0.9124	0.9888	0.9955	0.9921	0.9998	0.9568	...	0.9471	0.9799	0.8535	0.9860	0.9150	0.9778	0.9458	0.9346	0.9392	0.9755
6	CNN-char	0.8293	0.9176	0.8712	0.9559	0.9977	0.9910	0.9943	0.9997	0.9575	...	0.9245	0.9823	0.9582	0.9840	0.9709	0.9969	0.9532	0.9501	0.9511	0.9880
7	CNN-TA	0.8044	0.9242	0.8601	0.9580	0.9899	0.9944	0.9921	0.9999	0.9844	...	0.9567	0.9890	0.9430	0.9952	0.9684	0.9964	0.9574	0.9577	0.9569	0.9889
8	BERT (Ancient-Greek-BERT)	0.9553	0.9654	0.9603	0.9916	0.9955	0.9989	0.9972	1.0000	0.9807	...	0.9748	0.9952	0.9984	0.9964	0.9974	0.9999	0.9823	0.9818	0.9820	0.9955

- On the **train set**, the ranking in terms of **Macro-F1** and **Macro-PR-AUC** is:
BERT > BiLSTM-TA $\approx$ CNN-TA > CNN $\approx$ BiLSTM $\approx$ BiLSTM-char > MLP > Baseline.
- On the **dev set** **BERT > CNN-TA $\approx$ CNN-char > BiLSTM-char > BiLSTM $\approx$ CNN > MLP > Baseline.** While CNN-TA and CNN-char provide the strongest results among convolutional models and BiLSTM-char improves over the word-only BiLSTM, none of the sequence baselines match BERT’s macro-level performance. The per-word baseline continues to perform reasonably on frequent tags but struggles with rarer categories, whereas BERT maintains more balanced precision and recall across tags.
- On the **test set**, BERT again achieves the highest Macro-F1 and Macro-PR-AUC, demonstrating strong generalization beyond the training data. It consistently shows superior performance across both common and infrequent POS tags. CNN-TA and CNN-char slightly outperform their word-only counterparts and remain competitive with BiLSTM-based models, while the MLP remains below sequence models, and the baseline performs worst on rare categories.
- Overall, while CNN- and BiLSTM-based sequence models benefit from character-level information and temporal averaging, **BERT consistently provides the strongest and most stable performance across all splits.** These results highlight the effectiveness of pretrained contextual representations for POS tagging, particularly in capturing rare and morphologically complex tag distributions.

LLM Prompting

In addition to the BERT-based POS tagger, we conducted an exploratory experiment using a large language model (LLM) for **Ancient Greek POS tagging** via prompt-based inference. The approach constructs a structured, parseable prompt that specifies the UPOS tag set and enforces a strict output

format. When enabled, the prompt includes a small number of **few-shot demonstrations** randomly sampled from the training set, followed by unlabeled test sentences. A subset of 10 sentences is randomly sampled from the test set and presented to the LLM without gold annotations. The LLM's predictions are parsed and evaluated against the ground-truth POS tags using accuracy and macro-F1.

```
===== LLM RESULTS (10 test sentences) =====
Tokens evaluated: 156
Accuracy: 0.9231
Macro-F1: 0.9122
=====
sent_id=01 tokens= 26 accuracy=0.8462
sent_id=02 tokens= 6 accuracy=1.0000
sent_id=03 tokens= 19 accuracy=1.0000
sent_id=04 tokens= 18 accuracy=1.0000
sent_id=05 tokens= 38 accuracy=0.8947
sent_id=06 tokens= 9 accuracy=0.8889
sent_id=07 tokens= 14 accuracy=0.8571
sent_id=08 tokens= 5 accuracy=0.8000
sent_id=09 tokens= 3 accuracy=1.0000
sent_id=10 tokens= 18 accuracy=1.0000
```

On this limited sample of 10 test sentences, the LLM achieves competitive token-level performance, demonstrating that large language models can perform POS tagging reasonably well using only natural-language instructions and a small number of labeled examples.